

Колекції

Частина 3. Словники

Словник (змінюваний тип **dict**) — це колекція елементів, які організовані асоціативно. Елементи словника називають записами. Запис є парою ключ-значення, тобто доступ до значення конкретного елемента словника здійснюється не за його позицією (порядковим номером), як у випадку списків і кортежів, а за ключем.

Подання словників

Літералом словника в Python є послідовність пар ключ-значення, записаних через кому у фігурних дужках, при цьому ключ відділений від значення двокрапкою:

```
person = {'name': 'Роман', 'borned': 1976}
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25,
           6: 36, 7: 49, 8: 64, 9: 81, 10: 100}
pair = {'she':{'name':'Alice', 'from':'USA'},
        'he':{'name':'Bob', 'from':'Canada'}}
empty_dict = {}
```

Ключами зазвичай є цілі числа або рядки, але теоретично ключем може бути будь-який незмінюваний тип, наприклад, кортеж. Значення можуть бути якого завгодно типу.

Якщо ключем або значенням словника є вираз, то інтерпретатор обчислюватиме значення цього виразу:

```
dots1 = {'одна': '.', 'дві': '.'*2, 'три': '.'*3}
print(dots1)          # {'одна': '.', 'дві': '..', 'три': '...'}
```

або

```
dots2 = {'.':1, '.*2:2, '.*3:3}
print(dots2)          # {'.': 1, '.*2: 2, '.*3: 3}
```

Доступ до елементів словника

Доступ до значень елементів словника здійснюється подібно до списків, лише замість індекса у квадратних дужках вказується ключ:

```
print(squares[6])           # 36
print(pair['he']['from'])    # Canada
```

Оскільки словник — змінюваний тип даних, то його елементи можна змінювати:

```
rectangle = {'length': 2, 'width': 5}
rectangle['width'] = 4
print(rectangle)             # {'length': 2, 'width': 4}
```

Так само можна додати до словника новий елемент (пару ключ-значення):

```
rectangle['square'] = rectangle['length'] * rectangle['width']
print(rectangle)           # {'length': 2, 'width': 4, 'square': 8}
```

Якщо звернутися до елемента словника з неіснуючим ключем, з'явиться відповідне повідомлення про помилку:

```
print(rectangle['perimeter']) # KeyError: 'perimeter'
```

Такої помилки можна уникнути, виконавши перевірку з допомогою оператора `in`:

```
if 'perimeter' in rectangle:
    print(rectangle['perimeter'])
```

Проте ефективнішим вважається використання методу `get()`, який приймає два аргументи: 1) ключ; 2) значення, яке повертатиметься у випадку відсутності у словнику елемента з таким ключем:

```
print(rectangle.get('perimeter', None))
```

Функції для роботи зі списками та методи кортежів

Функція `len()` повертає кількість елементів словника:

```
points = {'Bob': 91, 'John': 60, 'Alice': 75}
print(len(points))           # 3
```

Функція `sorted()` повертає відсортований список ключів словника:

```
print(sorted(points))          # ['Alice', 'Bob', 'John']
```

Крім розглянутого вище методу `get()`, є ще низка інших (деякі з них часто використовуються разом з функцією `list`):

```
some_dict.pop(key)    # вилучає і повертає елемент за ключем
some_dict.clear()      # очищає словник
some_dict.copy()       # створює копію словника

some_dict.setdefault(key, value) # якщо ключа key немає у
# словнику, додає запис для цього ключа зі значенням value

list(some_dict.keys()) # список ключів
list(some_dict.values()) # список значень
list(some_dict.items()) # список кортежів (ключ, значення)
```

Поелементний обхід словника

Поелементний обхід словника зазвичай здійснюється одним з наступних двох способів:

```
# 1-й спосіб
for key in rectangle:
    print(f'{key} = {rectangle[key]}')

# 2-й спосіб
for key, value in rectangle.items():
    print(f'{key} = {value}')
```

Заповнення словників під час виконання програми

Як і списки, на практиці словники часто формуються під час виконання програми. Продемонструємо цей спосіб на прикладі програми, яка обчислює частоту входжень елементів у список:

```
from random import randint

# генеруємо список зі ста 0 і 1
bits = [randint(0, 1) for i in range(100)]
```

```
freq = {}                                # створюємо порожній словник
for bit in bits:                          # обходимо список поелементно
    if bit not in freq:                   # якщо ключа для біта немає у словнику,
        freq[bit] = 1                    # то це перше входження
    else:                                 # інакше
        freq[bit] += 1                   # збільшуємо кількість входжень на 1
print(freq)                              # див. виноски внизу сторінки1
```

Генератори словників

Словники підтримують генерування шляхом конструкції (інструкція `if` може бути опущена)

```
{ключ: вираз for ключ in колекція if умова}
```

Приклади генерування словника квадратів парних двоцифрових натуральних чисел:

```
d = {n: n**2 for n in range(10,100) if n%2 == 0}
d = {n: n**2 for n in range(10,100,2)}
```

У першому випадку парність числа перевіряється за остачею від ділення на 2, у другому перебір лише парних чисел досягається значенням кроку 2.

¹ Спробуйте також так 😊:
`import collections`
`print(collections.Counter(bits))`